

SAGE SHIELD CORE ENGINEERING

Sage Shield SIEM & Threat Intelligence API

Complete Integration, Out-of-Band Log Upload, Ingestion Pipeline & REST API Reference Manual

Document Version:	2.4.0 (Enterprise Production)
API Base Path:	/api/v1
Supported Providers:	Okta System Log, Microsoft Entra ID (Azure AD), Direct S3 Logs
Supported File Classes:	CSV, EVTX (Windows Event), JSON, Syslog (.log/.txt), PCAP, XLSX
Target Audience:	Frontend Engineers, SecOps Integrators, DevOps, Backend Clients

Executive Summary: Sage Shield is an enterprise security information and event management (SIEM) and AI-driven threat intelligence engine. This technical manual provides an exhaustive, step-by-step developer guide for all active Shield API endpoints, specifically detailing:

- **Data Source Integration:** Automated polling lifecycle, credential verification, AES-GCM encryption, and incremental checkpoint watermarking for Okta and Microsoft Entra ID.
- **Out-of-Band S3 Multipart Upload:** The complete multi-phase direct-to-storage architecture, S3 AWS Signature v4 policy construction, mandatory form field order, and client-side code implementations.
- **Asynchronous Ingestion Pipeline:** Background processing with Redis & Asyncq, automated file format detection, AST-indexed log parsing, and AI threat detector pipelines.
- **Log Parser Engine & Unified AST Search:** Custom regex/JSON/CSV parsers and the rich Abstract Syntax Tree (AST) search grammar.
- **Executive Security Intelligence:** Threat trends, identity risk posture, asset protection ratios, and live geo-IP attack vectors.

Table of Contents

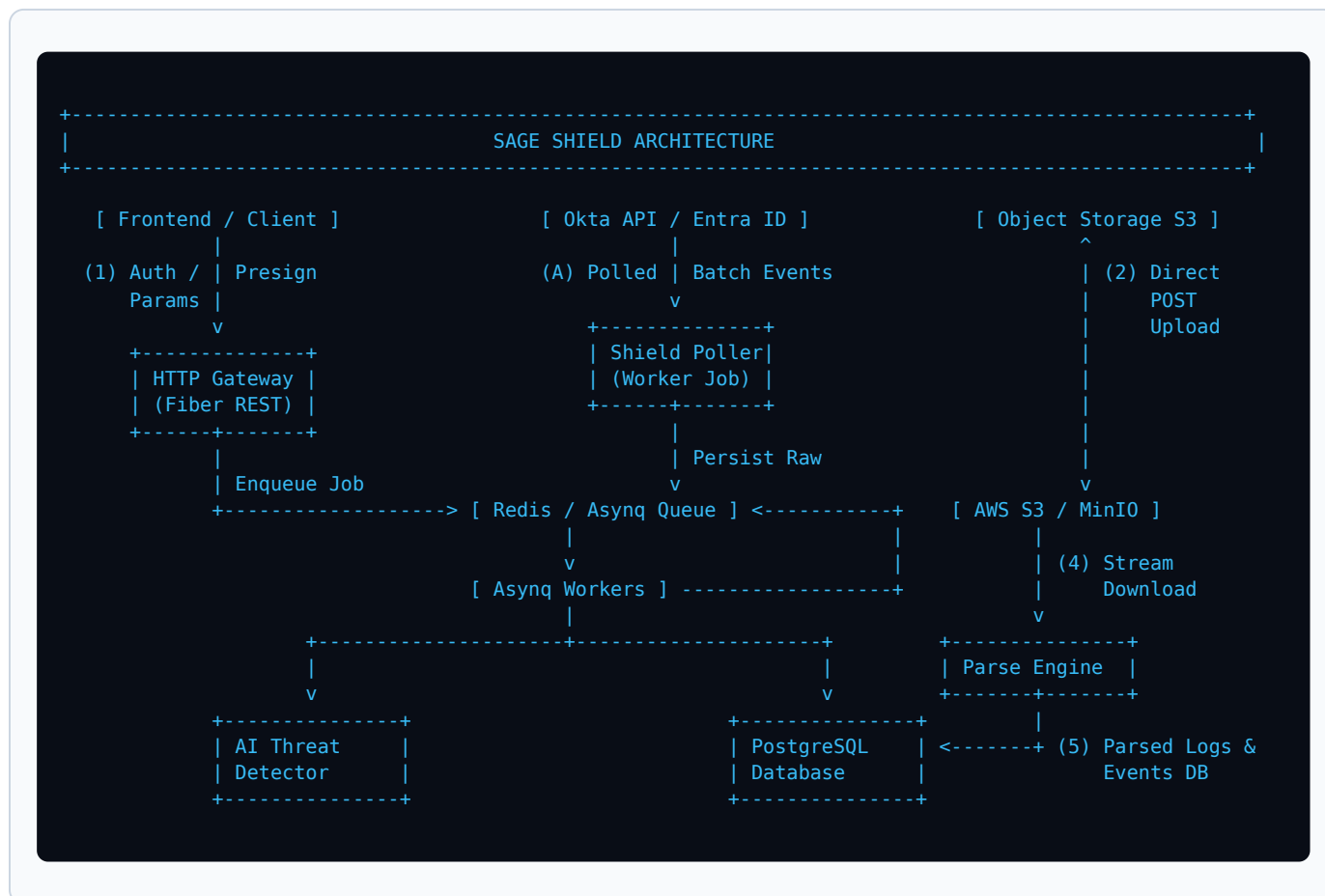
1. System Architecture & Authentication	Section 1
1.1 End-to-End System Ingestion Topology	1.1
1.2 Authentication, Multi-Tenancy & Session Headers	1.2
1.3 Unified API Response Envelope & Error Codes	1.3
2. Data Source Integration Engine (Okta & Entra ID)	Section 2
2.1 Integration & Checkpoint Synchronization Architecture	2.1
2.2 POST /api/v1/integrations (Connect Provider)	2.2
2.3 GET /api/v1/integrations/logs-data/sources (List Sources)	2.3
2.4 GET /api/v1/integrations/logs-data/sources/:id (Get Source Details)	2.4
2.5 POST /api/v1/integrations/logs-data/sources/:id/sync (Trigger Manual Sync)	2.5
2.6 POST /api/v1/integrations/logs-data/sources/:id/disconnect (Disconnect Source)	2.6
2.7 GET /api/v1/integrations/logs-data/sources/:id/logs (View Source Logs)	2.7
3. Ingestion Health Monitoring & Reporting	Section 3
3.1 GET /api/v1/integrations/logs-data/ingestion-health	3.1
3.2 POST /api/v1/integrations/logs-data/ingestion-health/refresh	3.2
3.3 GET /api/v1/integrations/logs-data/ingestion-health/volume	3.3
3.4 GET /api/v1/integrations/logs-data/ingestion-health/notifications	3.4
3.5 GET /api/v1/integrations/logs-data/ingestion-health/report	3.5
4. Out-of-Band Log Upload & Ingestion Pipeline (Deep Dive)	Section 4
4.1 Complete Multi-Phase Upload Flow & Sequence Diagram	4.1
4.2 Phase 1: POST /api/v1/integrations/logs-data/upload/presign	4.2
4.3 Phase 2: Direct Multipart Form POST to S3 (Outside the System API)	4.3
4.4 Phase 3: POST /api/v1/integrations/logs-data/upload/complete	4.4
4.5 Phase 4: Asynchronous Worker Pipeline & AI Threat Detection	4.5
4.6 Full Multi-Language Implementation Snippets (cURL, JS, Python, Go)	4.6
5. Custom Log Parser Management Engine	Section 5
5.1 Parser Lifecycle & Grammar Architecture	5.1

5.2 Parser Management Endpoints Reference (Summary, List, Create, Update)	5.2
5.3 Parser Testing, Preview, Enable/Disable, and Validation Endpoints	5.3
5.4 Import, Export & Sample Logs Endpoints	5.4
6. Unified Security Events & AST Query Engine	Section 6
6.1 AST Search Syntax Grammar & Query Expressions	6.1
6.2 GET /api/v1/events/logs (AST Query & Multi-Field Search)	6.2
6.3 GET /api/v1/events/logs/:id (Security Event Detail)	6.3
6.4 GET /api/v1/events/threats/summary	6.4
7. Executive Dashboard & Threat Intelligence Endpoints	Section 7
7.1 Security Posture Score, Identity Health & Asset Protection	7.1
7.2 Threat Intelligence Feeds, Active Incidents & Compliance Indicators	7.2
7.3 Threat Trends & Geo-IP Threat Distribution	7.3

1. System Architecture & Authentication

1.1 End-to-End System Ingestion Topology

Sage Shield SIEM operates on an event-driven, decoupled micro-architecture designed for sub-second log ingestion, high concurrency, and resilient third-party integrations.



The system separates two primary ingestion pipelines:

- Cloud Provider Polling Pipeline:** The backend worker periodically polls third-party APIs (Okta System Log, Entra ID) using encrypted credentials, tracks the ingestion watermark (checkpoint), normalizes raw events, and stores them in PostgreSQL.
- Direct-to-S3 File Upload Pipeline:** Massive log files (EVTX, CSV, PCAP, Syslog) bypass the API Gateway entirely via AWS S3 Presigned POST forms. Once uploaded, background workers stream the file, detect the schema, parse structured records into `parsed_logs`, and concurrently dispatch the stream to the AI Threat Detector.

1.2 Authentication, Multi-Tenancy & Session Headers

All Shield API endpoints operate inside an authenticated tenant perimeter enforced by `middlewares.AuthMiddleware`.

 Multi-Tenant Context Resolution

The server resolves the active user and tenant organization automatically through session storage (or bearer tokens). The user's `organizationID` and `userID` are securely extracted and injected into the database queries, guaranteeing strict tenant isolation.

Every HTTP request must provide valid session cookies or authorization headers:

```
GET /api/v1/events/logs HTTP/1.1
Host: api.sage.security
Authorization: Bearer <JWT_TOKEN_OR_SESSION_COOKIE>
X-Request-ID: req_8f9e4468-5e6b-4aa7-ba91-76537fe49afe
Content-Type: application/json
```

1.3 Unified API Response Envelope & Error Codes

All endpoints conform to a standardized JSON response envelope:

```
{
  "status": 200,
  "message": "Data retrieved successfully",
  "data": { ... },
  "error": null
}
```

HTTP Status	Error Code / Message	Description & Remediation
200 OK	SUCCESS	Request completed successfully.
201 Created	CREATED	Resource created or asynchronous background job successfully queued.
400 Bad Request	BAD_REQUEST / INVALID_ID	Malformed JSON body, missing mandatory fields, or invalid UUID.
401 Unauthorized	UNAUTHORIZED	Missing, expired, or invalid session or authentication token.
403 Forbidden	FORBIDDEN	User does not have required permissions or does not own the requested file.
404 Not Found	NOT_FOUND	Target data source, parser, log file, or security event does not exist.
422 Unprocessable	VALIDATION_FAILED	Payload failed schema validation (e.g. invalid URL, unsupported provider).
500 Server Error	INTERNAL_SERVER_ERROR	Internal database, S3, or worker queue failure. Check server logs.

2. Data Source Integration Engine (Okta & Entra ID)

2.1 Integration & Checkpoint Synchronization Architecture

Sage Shield connects to enterprise identity providers using pull-based polling. When an integration is added:

1. **Credential Verification:** The server calls the provider's API directly (e.g., Okta `/api/v1/logs?limit=1` or Microsoft Graph `/v1.0/auditLogs/signIns`) to verify token validity before persisting.
2. **AES-GCM Encryption:** Sensitive API keys, client secrets, and tokens are encrypted with AES-256-GCM before database insertion.
3. **Watermarking & Checkpoints:** The background worker maintains a `last_checkpoint` timestamp string in RFC-3339 format. Subsequent poll iterations only query events newer than this watermark, preventing duplicate ingestion.

POST /api/v1/integrations

Register & Connect Data Source

Connects an enterprise identity provider (Okta or Microsoft Entra ID) for continuous background log polling.

Request Body Schema

Field	Type	Requirement	Description
name	string	REQUIRED	Human-readable label for the integration (e.g. "Production Okta Cluster").
provider	string	REQUIRED	Target provider. Must be "okta" or "entra".
connection_type	string	REQUIRED	Connection mode. Typically "polling".
okta	object	OPTIONAL	Configuration required when provider = "okta".
okta.domain	string	REQ IF OKTA	Full Okta base URL (e.g. "https://company.okta.com").
okta.token	string	REQ IF OKTA	Okta SSWS API Token with okta.logs.read scope.
entra	object	OPTIONAL	Configuration required when provider = "entra".
entra.tenant_id	string	REQ IF ENTRA	Azure / Microsoft Entra Directory (Tenant) ID.
entra.client_id	string	REQ IF ENTRA	Registered Application (Client) ID.
entra.client_secret	string	REQ IF ENTRA	Client Secret value.

Example Request: Okta Integration

```
{
  "name": "Corporate Okta Production",
  "provider": "okta",
  "connection_type": "polling",
  "okta": {
    "domain": "https://company.okta.com",
    "token": "00abc123DEF456_secret_token"
  }
}
```

Example Request: Microsoft Entra ID Integration

```
{
  "name": "Azure AD / Entra Tenant",
  "provider": "entra",
  "connection_type": "polling",
  "entra": {
    "tenant_id": "8f9e4468-5e6b-4aa7-ba91-76537fe49afe",
    "client_id": "123e4567-e89b-12d3-a456-426614174000",
    "client_secret": "k7Q8-abcDEF_secretValue"
  }
}
```

Example Response (201 Created)

```
{
  "status": 201,
  "message": "INTEGRATION_CREATED",
  "data": null
}
```

GET /api/v1/integrations/logs-data/sources

List Connected Data Sources

Returns a paginated list of all active, paused, and uploaded data sources configured for the organization.

Query Parameters

Parameter	Type	Default	Description
page	integer	1	Page number for pagination.
page_size	integer	25	Items per page (max 100).
status	string	—	Filter by status: active , warning , error , inactive .
type	string	—	Filter by type: polling , log , webhook .
search	string	—	Full-text search across source name and description.

Example Response (200 OK)

```
{
  "status": 200,
  "message": "Sources retrieved",
  "data": {
    "items": [
      {
        "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
        "name": "Corporate Okta Production",
        "type": "polling",
        "provider": "okta",
        "status": "active",
        "events_today": 14250,
        "total_events": 892010,
        "last_event_at": "2026-08-25T21:40:00Z",
        "last_sync_at": "2026-08-25T21:42:00Z",
        "error_count": 0,
        "delayed_by_minutes": 0,
        "last_checkpoint": "2026-08-25T21:40:00Z",
        "created_at": "2026-08-01T09:00:00Z"
      }
    ],
    "total": 1,
    "page": 1,
    "page_size": 25
  }
}
```


GET

/api/v1/integrations/logs-data/sources/:id

Get Data Source Details

Fetches metadata, operational status, event counters, and checkpoint information for a single data source.

Path Parameters: `id` (UUID) — The Data Source unique identifier.

POST

/api/v1/integrations/logs-data/sources/:id/sync

Trigger Manual Sync Job

Enqueues an immediate on-demand polling job for the specified data source. Useful for real-time validation or testing newly configured credentials.

Example Response (200 OK)

```
{
  "status": 200,
  "message": "Sync queued",
  "data": {
    "job_id": "e09218e8-3a9d-4780-87ef-7e9bbfb91d24",
    "status": "queued",
    "source_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  }
}
```

POST

/api/v1/integrations/logs-data/sources/:id/disconnect

Disconnect Data Source

Transitions the data source status to `inactive` and disables scheduled polling jobs. All previously ingested security events and historical logs are preserved.

GET

/api/v1/integrations/logs-data/sources/:id/logs

Retrieve Logs by Source

Fetches ingested security logs scoped strictly to the specified data source.

Query Parameters

Parameter	Type	Description
<code>event_type</code>	string	Filter by event type (e.g. <code>"user.authentication.login"</code>).
<code>severity</code>	string	Filter by severity: <code>low</code> , <code>medium</code> , <code>high</code> , <code>critical</code> .
<code>start_time</code>	string (RFC3339)	Lower bound timestamp (e.g. <code>"2026-08-25T00:00:00Z"</code>).
<code>end_time</code>	string (RFC3339)	Upper bound timestamp.
<code>page</code> , <code>page_size</code>	integer	Pagination controls.

3. Ingestion Health Monitoring & Reporting

GET

/api/v1/integrations/logs-data/ingestion-health

Get Ingestion Health Summary

Returns organization-wide ingestion metrics, source operational states, event throughput, delay alerts, and error metrics.

Example Response (200 OK)

```
{
  "status": 200,
  "message": "Ingestion health retrieved",
  "data": {
    "total_events_today": 452090,
    "total_events_24h": 1240900,
    "active_sources_count": 8,
    "delayed_sources_count": 1,
    "error_sources_count": 0,
    "average_ingestion_latency_sec": 1.4,
    "pipeline_status": "healthy"
  }
}
```

POST

/api/v1/integrations/logs-data/ingestion-health/refresh

Refresh Health Metrics

Queues a background job to recalculate ingestion counters, query delays across external APIs, and refresh health cache.

GET

/api/v1/integrations/logs-data/ingestion-health/volume

Get Ingestion Volume Time-Series

Returns aggregated ingestion volume over time for chart rendering.

Query Parameters

Parameter	Type	Default	Description
interval	string	"hour"	Time bucket: "minute", "hour", "day".
source_id	string (UUID)	—	Optional filter for a specific data source.
start_time	string (RFC3339)	24h ago	Start timestamp.
end_time	string (RFC3339)	Now	End timestamp.

GET

/api/v1/integrations/logs-data/ingestion-health/notifications

Get Ingestion Alerts & Notifications

Returns active warnings and alerts regarding ingestion anomalies (e.g., token expiration, sudden drop in event volume, schema changes in log sources).

GET

/api/v1/integrations/logs-data/ingestion-health/report

Download Ingestion Health Report

Generates and exports an ingestion audit report as a downloadable binary file.

Query Parameters

Parameter	Type	Default	Description
format	string	"csv"	Export format: "csv", "pdf", "json".
start_time	string (RFC3339)	—	Report window start.
end_time	string (RFC3339)	—	Report window end.

Response Headers: Content-Type: application/octet-stream, Content-Disposition: attachment; filename=ingestion_report_2026-08-25.csv.

4. Out-of-Band Log Upload & Ingestion Pipeline (Deep Dive)

4.1 Complete Multi-Phase Upload Flow & Sequence Diagram

To support high-throughput, multi-gigabyte log uploads (such as Windows EVTX event logs, firewall PCAP dumps, or massive historical CSV archives) without overwhelming API Gateway threads or encountering HTTP timeout limits, Sage Shield utilizes a **Direct-to-S3 Presigned POST architecture**.



4.2 Phase 1: Request Presigned Upload Policy

POST

/api/v1/integrations/logs-data/upload/presign

Generate S3 Presigned Upload Policy

Initiates a log upload session by validating file constraints, creating a `pending` log record, and returning an S3 Presigned POST object containing AWS Signature v4 authorization fields.

Supported File Classes & Extensions

Class	MIME Types	Allowed Extensions	Target S3 Prefix
csv	text/csv	.csv	uploads/pending/csv/
windows_event	application/octet-stream	.evt , .evtx	uploads/pending/windows_event/
json	application/json	.json	uploads/pending/json/
syslog	text/plain , text/yaml	.log , .txt , .yaml , .yaml	uploads/pending/syslog/
pcap	application/vnd.tcpdump.pcap	.pcap	uploads/pending/pcap/
compressed	application/gzip , application/zip	.gz , .zip , .tar	uploads/pending/compressed/

Request Body Schema

```
{  "filename": "windows_security_ad_events.evtx",  "contentType": "application/octet-stream",  "size": 52428800 // File size in bytes (e.g. 50 MB)}
```

Response Body (200 OK)

```
{
  "status": 200,
  "message": "PresignURL created",
  "data": {
    "key": "uploads/pending/windows_event/428d4fcc-bb87-4aa7-8d2a-199480112345/req-17246244000000000000.evtx",
    "expires_at": "2026-08-25T22:00:00Z",
    "post": {
      "url": "https://sage-shield-bucket.s3.us-east-1.amazonaws.com",
      "fields": {
        "key": "uploads/pending/windows_event/428d4fcc-bb87-4aa7-8d2a-199480112345/req-17246244000000000000.evtx",
        "Content-Type": "application/octet-stream",
        "policy": "eyJleHBpcmF0aW9uIjoIMjAyNi0wOC0yNVQyMjo...",
        "x-amz-algorithm": "AWS4-HMAC-SHA256",
        "x-amz-credential": "AKIAIOSFODNN7EXAMPLE/20260825/us-east-1/s3/aws4_request",
        "x-amz-date": "20260825T214500Z",
        "x-amz-signature": "a3f58b0918c728519d...",
        "x-amz-meta-expected-class": "windows_event",
        "x-amz-meta-original-name": "windows_security_ad_events.evtx",
        "x-amz-meta-upload-source": "siem"
      }
    }
  }
}
```

4.3 Phase 2: Direct Multipart Form POST to S3 (Outside the System API)

In this phase, the client application conducts an **out-of-band HTTP POST** request directly to the S3 bucket URL (`post.url`). This interaction is completely separate from the Sage backend API.

⚠ CRITICAL S3 MULTIPART FORM RULES

AWS S3 enforces strict cryptographic validation on presigned form uploads. Failure to follow these three rules will cause S3 to reject the upload with `403 Forbidden` or `400 Bad Request` :

1. **Field Ordering Requirement:** All authentication and policy key-value pairs from `post.fields` must be appended to the form body **FIRST**.
2. **The `file` field MUST be LAST:** The binary file stream (field name must be `file`) **MUST BE THE FINAL ENTRY** in the multipart payload. S3 stops evaluating policy fields as soon as it encounters the file stream.
3. **DO NOT Send Authorization Headers to S3:** Do not include your application JWT token or Cookie headers when posting to S3. S3 will fail if unknown authorization headers are present.

Direct S3 POST Structure

```

POST / HTTP/1.1
Host: sage-shield-bucket.s3.us-east-1.amazonaws.com
Content-Type: multipart/form-data; boundary=---WebKitFormBoundary7MA4YWxkTrZu0gW

-----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="key"

uploads/pending/windows_event/428d4fcc-bb87-4aa7-8d2a-199480112345/req-17246244000000000000.evtx
-----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="Content-Type"

application/octet-stream
-----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="policy"

eyJleHBpcmF0aW9uIjoimjAyNi0wOC0yNVQyMjo...
-----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="x-amz-algorithm"

AWS4-HMAC-SHA256
-----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="x-amz-credential"

AKIAIOSFODNN7EXAMPLE/20260825/us-east-1/s3/aws4_request
-----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="x-amz-date"

20260825T214500Z
-----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="x-amz-signature"

a3f58b0918c728519d...
-----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="x-amz-meta-expected-class"

windows_event
-----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="x-amz-meta-original-name"

windows_security_ad_events.evtx
-----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="x-amz-meta-upload-source"

siem
-----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="file"; filename="windows_security_ad_events.evtx"
Content-Type: application/octet-stream

[BINARY EVENT LOG STREAM DATA]
-----WebKitFormBoundary7MA4YWxkTrZu0gW--

```

Direct S3 Response

Upon successful upload, AWS S3 responds with `HTTP/1.1 204 No Content` (or `200 OK`) and returns an `ETag` header:

```

HTTP/1.1 204 No Content
Date: Tue, 25 Aug 2026 21:46:12 GMT
ETag: "7f5d37276b649790cfd8f11ee5946e02"
x-amz-request-id: 98A7B6C5D4E3F210
Server: AmazonS3

```

 Capturing the ETag Header

Extract the `ETag` header value from the S3 response (including or excluding surrounding quotes, e.g. `"7f5d37276b649790cf8f11ee5946e02"`). You must supply this value in Phase 3 to confirm the upload integrity.

4.4 Phase 3: Confirm Upload & Enqueue Processing

POST

/api/v1/integrations/logs-data/upload/complete

Confirm Upload & Trigger Ingestion

Notifies the Sage Shield backend that the file has landed in S3. The server validates user ownership, issues an S3 `HeadObject` API call to verify the ETag and byte size, resolves the Data Source, marks the record as `confirmed`, and enqueues an asynchronous ingestion task.

Request Body Schema

Field	Type	Requirement	Description
<code>key</code>	string	REQUIRED	Exact S3 object key returned by <code>/upload/presign</code> .
<code>etag</code>	string	REQUIRED	ETag header returned by S3 (e.g. <code>"7f5d37276b..."</code>).
<code>metadata</code>	object	REQUIRED	Contextual metadata for parser routing and categorization.
<code>metadata.source_type</code>	string	REQUIRED	Source identifier (e.g. <code>"windows_security"</code> , <code>"nginx"</code> , <code>"firewall"</code>).
<code>metadata.source_id</code>	string (UUID)	OPTIONAL	Existing Data Source ID to attach these logs to. If omitted, a new source is auto-created.
<code>metadata.description</code>	string	OPTIONAL	Description of the uploaded log batch.
<code>metadata.category</code>	string	OPTIONAL	Log domain category (e.g. <code>"active_directory"</code> , <code>"endpoint"</code>).
<code>metadata.app_or_context</code>	string	OPTIONAL	Application or cluster context.
<code>metadata.host</code>	string	OPTIONAL	Origin host or node name (e.g. <code>"DC-PRIMARY-01"</code>).
<code>metadata.index_name</code>	string	OPTIONAL	Index grouping name.

Example Request

```
{
  "key": "uploads/pending/windows_event/428d4fcc-bb87-4aa7-8d2a-199480112345/req-1724624400000000000.evtx",
  "etag": "\"7f5d37276b649790cfd8f11ee5946e02\"",
  "metadata": {
    "source_type": "windows_security",
    "description": "Primary Domain Controller Security Logs",
    "category": "active_directory",
    "app_or_context": "domain_controllers",
    "host": "DC-PRIMARY-01",
    "index_name": "win_ad_sec"
  }
}
```

Example Response (200 OK)

```
{
  "status": 200,
  "message": "UPLOAD_COMPLETE",
  "data": {
    "id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
    "organization_id": "8f9e4468-5e6b-4aa7-ba91-76537fe49afe",
    "source_id": "123e4567-e89b-12d3-a456-426614174000",
    "s3_key": "uploads/pending/windows_event/428d4fcc-bb87-4aa7-8d2a-199480112345/req-1724624400000000000.evtx",
    "file_class": "windows_event",
    "source_type": "windows_security",
    "status": "confirmed",
    "created_at": "2026-08-25T21:45:00Z",
    "updated_at": "2026-08-25T21:46:15Z"
  }
}
```

4.5 Phase 4: Asynchronous Worker Pipeline & AI Threat Detection

Once confirmed, the Asynq task handler (`HandleProcessLogFile`) executes two concurrent background pipelines via Golang `errgroup` :

1. Parsing & Ingestion Pipeline:

- Streams the binary object from S3.
- Sniffs file magic bytes to automatically select the optimal parser (CSV, EVTX, JSON, Syslog, PCAP).
- Parses each raw event, structures timestamps, and bulk-inserts entries into the `parsed_logs` table.
- Transforms parsed entries into `SecurityEvent` domain entities and bulk-inserts them into the `security_events` table.

2. AI Threat Intelligence Pipeline:

- Dispatches the file stream to the AI threat detection service (`threatDetector.SubmitLogFileForAnalysis`).
- Performs deep anomaly classification, identifies suspicious privilege escalations, failed brute-force clusters, and generates actionable security incident objects.

4.6 Full Multi-Language Implementation Code

Option A: cURL / Bash Script

```
#!/usr/bin/env bash
set -e

FILENAME="windows_security_events.evtx"
FILESIZE=$(stat -c%s "$FILENAME")
API_URL="https://api.sage.security/api/v1"
SESSION_COOKIE="session=s%3A..."

# 1. Request Presigned Policy
PRESIGN_RESP=$(curl -s -X POST "$API_URL/integrations/logs-data/upload/presign" -H "Cookie: $SESSION_COOKIE" -H "Content-Type: application/json" -d '{"filename": "$FILENAME", "contentType": "application/octet-stream", "size": $FILESIZE}')

S3_URL=$(echo $PRESIGN_RESP | jq -r '.data.post.url')
KEY=$(echo $PRESIGN_RESP | jq -r '.data.key')

# 2. Extract fields & POST directly to S3 (Notice FILE is LAST)
ETAG=$(curl -s -i -X POST "$S3_URL" -F "key=$(echo $PRESIGN_RESP | jq -r '.data.post.fields.key')" -F "Content-Type=$(echo $PRESIGN_RESP | jq -r '.data.post.fields[\"Content-Type\"]')" -F "policy=$(echo $PRESIGN_RESP | jq -r '.data.post.fields.policy')" -F "x-amz-algorithm=$(echo $PRESIGN_RESP | jq -r '.data.post.fields[\"x-amz-algorithm\"]')" -F "x-amz-credential=$(echo $PRESIGN_RESP | jq -r '.data.post.fields[\"x-amz-credential\"]')" -F "x-amz-date=$(echo $PRESIGN_RESP | jq -r '.data.post.fields[\"x-amz-date\"]')" -F "x-amz-signature=$(echo $PRESIGN_RESP | jq -r '.data.post.fields[\"x-amz-signature\"]')" -F "x-amz-meta-expected-class=$(echo $PRESIGN_RESP | jq -r '.data.post.fields[\"x-amz-meta-expected-class\"]')" -F "x-amz-meta-original-name=$(echo $PRESIGN_RESP | jq -r '.data.post.fields[\"x-amz-meta-original-name\"]')" -F "x-amz-meta-upload-source=$(echo $PRESIGN_RESP | jq -r '.data.post.fields[\"x-amz-meta-upload-source\"]')" -F "file=@$FILENAME" | grep -i etag | awk '{print $2}' | tr -d '
')

# 3. Complete Upload
curl -s -X POST "$API_URL/integrations/logs-data/upload/complete" -H "Cookie: $SESSION_COOKIE" -H "Content-Type: application/json" -d '{"key": "$KEY", "etag": $ETAG, "metadata": {"source_type": "windows_security", "category": "active_directory"}}'
```

Option B: JavaScript / TypeScript (Browser or Node.js)

```
async function uploadLogFile(file) {  
  // Step 1: Request Presigned URL  
  const presignRes = await fetch('/api/v1/integrations/logs-data/upload/presign', {  
    method: 'POST',  
    headers: { 'Content-Type': 'application/json' },  
    body: JSON.stringify({  
      filename: file.name,  
      contentType: file.type || 'application/octet-stream',  
      size: file.size,  
    }),  
  });  
  const { data } = await presignRes.json();  
  
  // Step 2: Build Multipart Form Data (Fields MUST precede the file)  
  const formData = new FormData();  
  Object.entries(data.post.fields).forEach(([k, v]) => {  
    formData.append(k, v);  
  });  
  // Crucial rule: file must be the last field appended  
  formData.append('file', file);  
  
  // Step 3: Direct POST to S3 (Do NOT add Auth headers)  
  const s3Res = await fetch(data.post.url, {  
    method: 'POST',  
    body: formData,  
  });  
  
  if (!s3Res.ok && s3Res.status !== 204) {  
    throw new Error(`S3 Upload failed with status ${s3Res.status}`);  
  }  
  
  const etag = s3Res.headers.get('ETag');  
  
  // Step 4: Confirm Upload with Sage Shield API  
  const confirmRes = await fetch('/api/v1/integrations/logs-data/upload/complete', {  
    method: 'POST',  
    headers: { 'Content-Type': 'application/json' },  
    body: JSON.stringify({  
      key: data.key,  
      etag: etag,  
      metadata: {  
        source_type: 'windows_security',  
        category: 'active_directory',  
        description: 'Uploaded via SIEM Web Console',  
      },  
    }),  
  });  
  
  return await confirmRes.json();  
}
```

Option C: Python (Requests)

```
import requests

def upload_log_file(filepath, session_cookie):
    filename = os.path.basename(filepath)
    size = os.path.getsize(filepath)

    # 1. Presign
    headers = {"Cookie": f"session={session_cookie}"}
    r = requests.post("https://api.sage.security/api/v1/integrations/logs-data/upload/presign",
                      headers=headers,
                      json={"filename": filename, "contentType": "application/octet-stream", "size": size})
    presign = r.json()["data"]

    # 2. POST to S3
    fields = presign["post"]["fields"]
    s3_url = presign["post"]["url"]

    with open(filepath, "rb") as f:
        files = {"file": (filename, f, "application/octet-stream")}
        s3_resp = requests.post(s3_url, data=fields, files=files)

    etag = s3_resp.headers.get("ETag")

    # 3. Complete
    complete_payload = {
        "key": presign["key"],
        "etag": etag,
        "metadata": {"source_type": "windows_security", "category": "active_directory"}
    }
    res = requests.post("https://api.sage.security/api/v1/integrations/logs-data/upload/complete",
                        headers=headers, json=complete_payload)
    return res.json()
```

5. Custom Log Parser Management Engine

5.1 Parser Lifecycle & Grammar Architecture

Sage Shield supports dynamic user-defined parsers to extract structured key-value entities from raw log strings. Supported parser types include:

- `regex`: Named regular expression capture groups mapped to standard SIEM fields.
- `json`: Extraction of nested JSON paths (e.g. `"actor.email" -> actor_email`).
- `csv`: Column delimiter and header indexing.
- `key_value`: Key-value pair delimiter parsing (e.g. `k1=v1 k2=v2`).
- `ai_nlp`: Zero-shot natural language parser extraction powered by machine learning models.

GET `/api/v1/integrations/logs-data/parsers/summary`

Get Parser Metrics Summary

Returns health metrics for the parser catalog, including active counts, 24h error rates, and last update timestamp.

GET

/api/v1/integrations/logs-data/parsers

List Custom Parsers

Returns a paginated list of custom parsers with filtering support.

Query Parameters

Parameter	Type	Description
status	string	Filter by status: active , warning , error , disabled .
parser_type	string	Filter by engine: regex , json , csv , key_value , ai_nlp .
search	string	Search across parser names, descriptions, and tags.
page , page_size	integer	Pagination parameters.

GET

/api/v1/integrations/logs-data/parsers/:id

Get Parser Specification

Fetches full configuration details, field mappings, parsing rules, and execution statistics for a specific parser.

POST

/api/v1/integrations/logs-data/parsers

Create Custom Parser

Creates a new parser definition with field mappings and extraction logic.

Example Request

```
{
  "name": "Nginx Access Log Parser",
  "description": "Extracts IP, status code, method, and URI from Nginx logs",
  "parser_type": "regex",
  "tags": ["web", "nginx", "http"],
  "logic": {
    "pattern": "(?P<ip>\\S+) - (?P<user>\\S+) \\[(?P<time>[^\"]+)]\" (?P<method>\\S+) (?P<uri>\\S+) \\S+\" (?P<status>\\d+) (?P<bytes>\\d+)"
  },
  "mappings": [
    { "source_field": "ip", "target_field": "ip_address", "type": "string" },
    { "source_field": "user", "target_field": "actor_username", "type": "string" },
    { "source_field": "method", "target_field": "event_type", "type": "string" }
  ]
}
```

PATCH

/api/v1/integrations/logs-data/parsers/:id

Update Parser Definition

Modifies logic, field mappings, tags, or status of an existing parser. Automatically creates an immutable audit version history before applying updates.

POST **/api/v1/integrations/logs-data/parsers/:id/test**

Test Parser Against Sample

Executes an existing parser against a provided log string or raw JSON payload and returns the normalized output and any parse errors.

```
{
  "sample_log": "192.168.1.100 - admin [25/Aug/2026:21:50:00 +0000] \"GET /admin/dashboard HTTP/1.1\" 200 4520"
}
```

POST **/api/v1/integrations/logs-data/parsers/preview**

Sandbox Preview Parser

Tests unsaved parser logic against sample input. Enables frontend parser builders to test regex patterns and field mappings in real time before persisting.

POST **/api/v1/integrations/logs-data/parsers/:id/enable**

Enable Custom Parser

Activates the parser for future automatic log normalization.

POST **/api/v1/integrations/logs-data/parsers/:id/disable**

Disable Custom Parser

Deactivates the parser without deleting it or altering historical parsed logs.

POST **/api/v1/integrations/logs-data/parsers/:id/validate**

Validate Parser against Historic Stream

Queues an asynchronous validation job testing the parser against the last 500 logs from its linked data source.

POST **/api/v1/integrations/logs-data/parsers/validate**

Batch Validate All Parsers

Queues system-wide validation across all active parsers to detect syntax breakage, schema drifts, or elevated error rates.

POST `/api/v1/integrations/logs-data/parsers/import`

Import Parser JSON

Imports a standardized parser JSON definition from backup, template repository, or community packs.

GET `/api/v1/integrations/logs-data/parsers/:id/export`

Export Parser JSON

Exports the complete parser configuration as portable JSON.

GET `/api/v1/integrations/logs-data/parsers/sample-logs`

Fetch Sample Logs for Testing

Returns sample unparsed and parsed log records filtered by `source_id` or `parser_id` to assist engineers when designing parser expressions.

6. Unified Security Events & AST Query Engine

6.1 AST Search Syntax Grammar & Query Expressions

The AST Query Engine (`ASTSearchService`) provides query capabilities across both API-pollled logs and S3 uploaded files via the `q` query parameter.

Query Expression	Target Field / Interpretation	Example Query
<code>"quoted phrase"</code>	Full-text substring search across message and payload.	<code>q="unauthorized access"</code>
<code>level=<SEVERITY></code>	Filter by log level / severity (<code>ERROR</code> , <code>WARN</code> , <code>INFO</code> , <code>DEBUG</code>).	<code>q='level=ERROR'</code>
<code>source=<UUID></code>	Filter by specific Data Source UUID.	<code>q='source=3fa85f64-5717-4562-b3fc-2c963f66afa6'</code>
<code>raw.<key>=<val></code>	Deep extraction query on JSON field inside raw payload.	<code>q='raw.ip_address=192.168.1.50'</code>
Compound Expression	Combines multiple AST clauses into a single unified search.	<code>q='level=ERROR "failed login" raw.user_id=usr_992'</code>

Searches and filters parsed logs and security events across all sources.

Query Parameters

Parameter	Type	Description
q	string	AST Query String (e.g. <code>level=ERROR "failed login"</code>). If provided, takes precedence.
source_id	string (UUID)	Filter by Data Source ID.
source	string	Filter by source name / provider (e.g. <code>"okta"</code> , <code>"windows_event"</code>).
event_type	string	Filter by event type (e.g. <code>"user.authentication.auth_via_mfa"</code>).
severity	string	Filter by severity: <code>low</code> , <code>medium</code> , <code>high</code> , <code>critical</code> .
actor_email	string	Filter by actor's email address.
ip_address	string	Filter by IP address.
start_time , end_time	string (RFC3339)	Time window filters.
page , page_size	integer	Pagination (default: <code>page=1</code> , <code>page_size=25</code> , max: <code>200</code>).

Example Response (200 OK)

```
{
  "status": 200,
  "message": "Logs retrieved",
  "data": {
    "items": [
      {
        "id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
        "source_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
        "source": "Corporate Okta Production",
        "event_type": "user.authentication.auth_via_mfa",
        "event_category": "authentication",
        "severity": "high",
        "actor_email": "secops@sage.security",
        "ip_address": "192.168.1.10",
        "geo_country": "United States",
        "geo_city": "San Francisco",
        "raw_payload": {
          "actor": { "id": "usr_991", "type": "User" },
          "outcome": { "result": "FAILURE", "reason": "INVALID_OTP" }
        },
        "normalized_payload": {
          "action": "mfa_challenge_failed",
          "user_id": "usr_991"
        },
        "parse_status": "success",
        "occurred_at": "2026-08-25T21:40:00Z",
        "ingested_at": "2026-08-25T21:40:02Z"
      }
    ],
    "total": 1,
    "page": 1,
    "page_size": 25
  }
}
```

GET **/api/v1/events/logs/:id**

Get Log Event Detail

Fetches full raw payload, normalized mapping, geolocational metadata, and parse diagnostic details for a specific security event ID.

GET **/api/v1/events/threats/summary**

Get Threats & Vulnerabilities Summary

Returns aggregate counts of detected threats and known vulnerabilities categorized by critical, high, medium, and low severities.

7. Executive Dashboard & Threat Intelligence Endpoints

GET **/api/v1/security-posture/score**

Overall Security Posture Score (Widget 1)

Returns the consolidated multi-pillar security score (0–100), weekly delta trend, and itemized pillar health scores.

```
{
  "status": 200,
  "message": "Security posture score retrieved",
  "data": {
    "overall_score": 88,
    "weekly_delta": 3,
    "description": "Security posture improved due to increased MFA coverage",
    "pending_recommendations": 4,
    "pillars": {
      "config_health": 92,
      "vulnerabilities": 85,
      "threat_coverage": 90,
      "response_readiness": 84
    }
  }
}
```

GET **/api/v1/identity-health/summary**

Identity & Access Health Summary (Widget 3)

Returns MFA adoption percentage, dormant identity counts, and elevated access risk metrics across Okta and Entra ID.

```
{
  "coverage_percentage": 94,
  "accounts_without_mfa": 12,
  "dormant_accounts": 5,
  "elevated_privileges": 18
}
```

GET **/api/v1/assets/protection-coverage**

Endpoint Protection Coverage (Widget 4)

Returns asset protection ratio comparing protected vs unprotected endpoints across monitored cloud and domain hosts.

GET **/api/v1/threat-intel/feeds/summary**

Threat Intelligence Feeds Summary (Widget 5)

Returns indicator ingestion velocity (indicators processed in the last 24h) and feed operational states.

GET

/api/v1/incidents

Active Security Incidents Feed (Widget 6)

Returns active incidents requiring analyst attention. Supports `?limit=10` query parameter.

GET

/api/v1/compliance/risk-indicators

Compliance Risk Indicators (Widget 8)

Returns itemized compliance control risks including unencrypted volumes, excessive permissions, and physical security flags.

GET

/api/v1/events/threats/asset-risk-distribution

Asset Threat Risk Distribution (Widget 7)

Returns asset-level threat distribution percentages for risk breakdown donut charts.

GET

/api/v1/events/threat-trends

Threat Severity Month-over-Month Trends (Widget 9)

Returns daily threat incident counts comparing the target month against the previous baseline period.

Query Parameters

Parameter	Type	Default	Description
current_month	string	Current Month	Target month format: <code>2026-08</code> , <code>August</code> , or <code>8</code> .
previous_month	string	Previous Month	Comparison baseline month format: <code>2026-07</code> , <code>July</code> , or <code>7</code> .

GET **/api/v1/events/geo-threats**

Live Geo-IP Threat Origins (Widget 10)

Returns geospatial attack origins with latitude/longitude coordinates, total threat volume, and top targeted assets for world map visualizations.

```
{
  "status": 200,
  "message": "Geo threat origins retrieved",
  "data": {
    "total_threats": 2841,
    "high_threat_region": "Eastern Europe",
    "most_targeted_asset": "Auth-Cluster-01",
    "origins": [
      { "country": "Romania", "lat": 45.9432, "lng": 24.9668, "count": 420 },
      { "country": "Brazil", "lat": -14.2350, "lng": -51.9253, "count": 315 }
    ]
  }
}
```

— END OF SAGE SHIELD API DOCUMENTATION —

For support, email support@sage.security